

HybridKernel: A Profiler-Driven Boundary-Fusion Gate for Hybrid Attention/SSM Serving

Anonymous authors

Paper under double-blind review

Abstract

Hybrid attention/state-space language models invite a systems question: do attention-to-SSM layer boundaries create avoidable conversion, materialization, launch, or locality overhead that a fused boundary operator could remove? We report a Mac-feasible pre-GPU gate for this question. Architecture maps show large boundary-crossing activation streams, but a runtime audit weakens the broad novelty claim because current vLLM hybrid SSM support already handles important state-layout and transfer paths. A threshold model says Granite 4.0 H needs roughly 25% genuinely avoidable boundary traffic at 60% recovery to clear a 3% proxy gain; Qwen3-Next needs 10.4% but is less directly matched to the Granite Mamba2 hypothesis. We therefore do not claim a kernel speedup. The contribution is a reproducible profiler gate, a fixed-request vLLM driver, a pre-registered profiler-analysis parser, and an artifact-completeness verifier with a promote/kill criterion for future native NVIDIA measurements.

1 Hypothesis

HybridKernel tests whether adjacent attention and SSM layers expose boundary-local overhead not already removed by serving systems. Candidate costs include layout conversion, residual-stream materialization, host launch gaps, cache-locality loss, or redundant memory traffic. The hypothesis is deliberately narrow: a useful kernel must compose with existing vLLM hybrid mechanisms rather than replace them.

2 Current Evidence

3 Profiler Gate

The next experiment is a native NVIDIA/vLLM run with Nsight Systems and Nsight Compute. It must show a distinct boundary-local overhead attributable to attention/SSM transitions, not warmup, graph capture, batching, or unrelated kernels. The runbook now specifies exactly how to reduce Nsight traces into parser fields: total step time, boundary-local time, matched non-boundary control time, and recoverable fraction. Promotion requires at least three repeated runs whose recoverable-gain upper bound clears 3%. The analysis parser computes

$$\text{gain}_{ub} = \frac{\max(\text{boundary ms} - \text{matched control ms}, 0)}{\text{total step ms}} \times \text{recoverable fraction}.$$

If repeated native summaries show less than 1% recoverable gain, the branch should be killed or shelved. Before any run is cited, the artifact verifier must also pass on the exact run directory, confirming environment metadata, Nsight Systems and Nsight Compute artifacts, profiling logs, the pre-registered readout questions, and at least three valid repeated metric rows for one model. This is an admissibility check only; it does not promote the method.

Artifact	Result	Decision
Architecture map	Granite has 8 boundaries and 20.0% boundary stream fraction; Qwen3-Next has 23 inferred boundaries and 47.9%.	Inspectable, not speed evidence.
Runtime audit	vLLM already handles hybrid state layout and transfer paths.	Broad novelty weakened.
Threshold model	Granite needs about 25.0% avoidable boundary traffic at 60% recovery for a 3% proxy gain.	Mac kernel work not justified.
Profiler driver	Fixed-request OpenAI-compatible driver dry-runs locally.	Native gate is runnable.
Analysis parser	JSON profiler summaries are reduced into avoidable-boundary share and recoverable-gain upper bound.	Decision rule is pre-registered.
Artifact verifier	Native run directories are checked for metadata, Nsight artifacts, logs, readout rows, and repeated metric rows.	Evidence must be review-complete before citation.

Table 1: HybridKernel evidence before native GPU profiling.

4 Claim Boundary

Allowed now: HybridKernel is a weakly alive profiler-driven systems branch with a runnable native measurement plan. Not allowed: throughput, latency, HBM, energy, occupancy, or vLLM speedup claims. If native profiling does not reveal separable boundary overhead, the branch should be killed.

5 Artifacts

- experimental/hybridkernel/phase2/nvidia_vllm_profiler_runbook.md
- experimental/hybridkernel/phase2/profiler_driver.py
- experimental/hybridkernel/phase2/analyze_profiler_metrics.py
- experimental/hybridkernel/phase2/check_profiler_run_artifacts.py
- experimental/hybridkernel/phase2/profiler_analysis_gate.md
- experimental/hybridkernel/phase2/pre_gpu_threshold_model.md
- experimental/hybridkernel/paper/colm_workshop_scaffold.md